

Informe de Desarrollo: Sweet Crush

Análisis, Diseño Algorítmico y Gestión Dinámica de Memoria

Sofia Arboleda Zapata - Jose Manuel Arias Toro

Universidad de Antioquia – Informática II

Septiembre de 2026

Índice

1. Análisis del problema y consideraciones para la alternativa de solución propuesta	2
1.1. Representación de las fichas	2
1.2. Selección de símbolos para la visualización	2
1.3. Cálculo de memoria requerida	2
1.4. Elección del tipo de dato para el almacenamiento físico	3
2. Esquema de las tareas definidas en el desarrollo de los algoritmos	3
2.1. Arquitectura modular	4
3. Algoritmos implementados	5
3.1. Lectura y escritura de una ficha a nivel de bits	5
3.2. Detección de combinaciones	6
3.3. Aplicación de gravedad	8
3.4. Resolución de cascadas	9
3.5. Modificación dinámica de la estructura (filas/columnas)	10
4. Problemas de desarrollo que afrontó	11
5. Evolución de la solución y consideraciones para la implementación	12
5.1. Optimización de la estructura de marcado	12

1. Análisis del problema y consideraciones para la alternativa de solución propuesta

1.1. Representación de las fichas

El enunciado exige que cada posición del tablero se represente utilizando exactamente 3 bits, ya que con esta cantidad es posible codificar $2^3 = 8$ combinaciones distintas, de las cuales 6 se requieren para diferenciar los seis tipos de fichas del juego y las 2 restantes quedan disponibles para representar estados adicionales (por ejemplo, una posición vacía).

La codificación adoptada asocia cada ficha con su equivalente decimal de forma secuencial, tal como se muestra en la siguiente tabla:

Código Binario	Decimal	Estado
000	0	Ficha A
001	1	Ficha B
010	2	Ficha C
011	3	Ficha D
100	4	Ficha E
101	5	Ficha F
110	6	Estado vacío

1.2. Selección de símbolos para la visualización

Para la representación visual de las fichas en consola se descartó el uso directo de las letras A–F propuestas como ejemplo en el enunciado (Figura 1), ya que algunas de ellas resultan visualmente confusas entre sí — en particular las letras E y F, señaladas explícitamente como poco recomendables incluso por el profesor, quien aclaró que su uso en el enunciado obedecía únicamente a fines ilustrativos y no constituía una recomendación de implementación.

Se exploraron alternativas con caracteres especiales (corazones, estrellas y símbolos similares), pero estos presentaron problemas de codificación y no se visualizaban correctamente en la consola. Por esta razón se optó finalmente por un conjunto de símbolos ASCII estándar (@ # \$ % & +), los cuales garantizan una representación consistente en cualquier consola, son claramente distinguibles entre sí y resultan visualmente más variados que el uso exclusivo de letras.

Código Binario	Símbolo	Estado
000	@	Ficha A
001	#	Ficha B
010	\$	Ficha C
011	%	Ficha D
100	&	Ficha E
101	+	Ficha F
110	.	Vacío / marcado para eliminación

El símbolo . (código 110) se utiliza para marcar transitoriamente las posiciones cuyas fichas fueron identificadas como parte de una combinación detectada y, por tanto, están a punto de ser eliminadas del tablero. Una vez procesada la eliminación, estas posiciones son ocupadas por la gravedad (fichas provenientes de posiciones superiores) o por fichas nuevas generadas aleatoriamente.

1.3. Cálculo de memoria requerida

Dado que cada byte de memoria contiene 8 bits, y cada ficha del tablero requiere exactamente 3 bits para su representación, la cantidad total de bits necesarios para un tablero de F filas por

C columnas es:

$$\text{bits totales} = F \times C \times 3$$

Para determinar cuántos bytes deben reservarse, se divide esta cantidad de bits entre 8. Sin embargo, dado que los bits de una ficha pueden quedar repartidos entre dos bytes consecutivos (como se ilustra en la Figura 3 del enunciado), no basta con truncar el resultado de la división: si la división no es exacta, es necesario reservar un byte adicional para poder alojar los bits sobrantes, pues de lo contrario no habría espacio físico suficiente para las fichas cuyos bits cruzan la frontera entre dos bytes. Esto se implementó mediante aritmética entera, evaluando el resto de la división:

```
1 int calcularCantidadBytes(int filas, int columnas) {
2     int cantidad_bits = filas * columnas * 3;
3     int cantidad_bytes = cantidad_bits / 8;
4     if (cantidad_bits % 8 != 0) {
5         cantidad_bytes += 1;
6     }
7     return cantidad_bytes;
8 }
```

$$\text{bytesReservados} = \left\lceil \frac{F \times C \times 3}{8} \right\rceil$$

pero se optó por resolverlo mediante división entera y el operador módulo (%), en lugar de utilizar la función `ceil()` de la biblioteca `<cmath>`, evitando así la conversión implícita entre tipos flotantes y enteros para un cálculo cuyo resultado siempre debe ser un número entero de bytes.

Ejemplo numérico. Para un tablero de 5×5 posiciones:

$$\text{bits totales} = 5 \times 5 \times 3 = 75 \text{ bits}$$

Al dividir entre 8: $75/8 = 9$ con un resto de 3 (ya que $9 \times 8 = 72$, y $75 - 72 = 3$). Como el resto es distinto de cero, se reserva un byte adicional, resultando en:

$$\text{bytesReservados} = 9 + 1 = 10 \text{ bytes}$$

Es decir, un tablero de $5 \times 5 = 25$ fichas, que requiere 75 bits, se almacena en 10 bytes (80 bits disponibles), de los cuales los últimos 5 bits del bloque reservado no corresponden a ninguna ficha y permanecen sin usar.

1.4. Elección del tipo de dato para el almacenamiento físico

Se seleccionó `unsigned char*` como tipo base para el bloque de memoria dinámica del tablero porque la representación exigida por el desafío opera directamente sobre bits individuales, y `unsigned char` es el tipo entero más pequeño disponible en C++ para el cual el lenguaje garantiza un tamaño de exactamente 8 bits. Usar un tipo de mayor tamaño, como `unsigned int` (típicamente 32 bits), habría significado manipular cuatro veces más bits de los necesarios en cada acceso a memoria y habría complicado innecesariamente el cálculo de la posición de cada ficha dentro de la secuencia continua de bits.

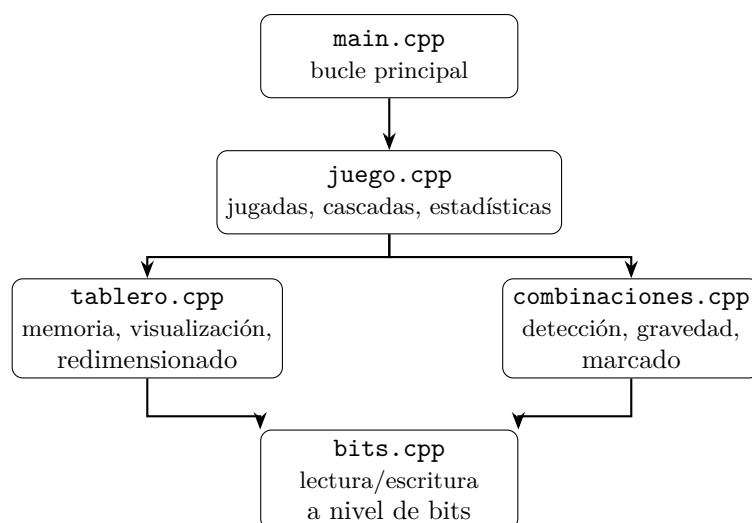
2. Esquema de las tareas definidas en el desarrollo de los algoritmos

El flujo general del programa se estructura en las siguientes etapas:

1. **Solicitud de dimensiones:** Se le pide al usuario el número de filas y columnas con los que desea jugar (`pedirDimensiones`).
2. **Inicialización del tablero:** Se reserva la memoria dinámica correspondiente y se generan fichas de forma aleatoria para llenar todas las posiciones (`inicializarJuego`, `generarFichasAleatorias`).
3. **Despliegue del menú de acciones:** El programa muestra el estado actual del tablero y solicita al usuario un número entero. El diseño reserva los valores negativos para comandos especiales de modificación estructural del tablero (`-1` salir, `-2` agregar fila, `-3` eliminar fila, `-4` eliminar columna, `-5` agregar columna), dejando libres los valores positivos para que el usuario indique directamente la fila donde se encuentra la ficha que desea eliminar (*“Eliminar una ficha (indique el número de fila donde está)”*). De esta forma, un mismo campo de entrada numérica distingue de forma no ambigua entre una jugada normal y un comando de modificación del tablero, sin necesidad de un menú separado.
4. **Ejecución de la jugada:** Si el usuario seleccionó una fila válida, se le solicita además la columna, y se procede a eliminar la ficha en esa posición (`ejecutarJugadaUsuario`, `procesarJugada`).
5. **Resolución de cascadas:** Tras cada eliminación (sea por jugada del usuario o por una modificación estructural), el sistema aplica gravedad, rellena los espacios vacíos y vuelve a verificar si se formaron nuevas combinaciones, repitiendo este proceso mientras sigan apareciendo combinaciones (`resolverCascadas`). Cada iteración de este ciclo cuenta como una *cascada*, entendida como toda combinación (horizontal o vertical) que se produce como consecuencia del desplazamiento de fichas tras una caída por columna.
6. **Actualización de estadísticas:** Se actualiza el puntaje del jugador, otorgando 10 puntos por cada combinación detectada y 1 punto por cada ficha eliminada (`actualizarEstadisticas`).

2.1. Arquitectura modular

La solución se organiza en cuatro módulos con responsabilidades claramente separadas, siguiendo una jerarquía de dependencias en la que cada capa solo conoce a la capa inmediatamente inferior:



El módulo `bits.cpp` constituye la capa más baja de la arquitectura: es el único responsable de traducir una posición lógica (fila, columna) a su ubicación física dentro de la secuencia de bits, y de leer o escribir los 3 bits correspondientes a una ficha. Ningún otro módulo manipula

bits directamente; todos acceden al tablero exclusivamente a través de las funciones `leerFicha` y `escribirFicha`.

Sobre esa base se apoyan dos módulos independientes entre sí: `tablero.cpp`, encargado del ciclo de vida de la memoria dinámica del tablero (creación, destrucción, visualización y redimensionamiento estructural), y `combinaciones.cpp`, encargado de la lógica de detección de combinaciones, marcado, eliminación y aplicación de gravedad.

Finalmente, `juego.cpp` coordina ambos módulos para implementar el flujo completo de una partida (jugadas del usuario, resolución de cascadas, estadísticas), y `main.cpp` contiene únicamente el bucle de interacción con el usuario.

3. Algoritmos implementados

3.1. Lectura y escritura de una ficha a nivel de bits

Estas dos funciones constituyen el núcleo de la manipulación bitwise de la solución: son las únicas encargadas de traducir una posición lógica del tablero en su ubicación física dentro del bloque continuo de bits, y de extraer o modificar únicamente los 3 bits correspondientes sin afectar el resto de la información almacenada.

A partir del índice lógico de la ficha (calculado como $\text{fila} \times \text{columnas} + \text{columna}$), se determina el `bit_inicial` multiplicando dicho índice por 3. De ahí se obtienen el byte donde comienza la ficha (`bit_inicial / 8`) y el desplazamiento dentro de ese byte (`bit_inicial % 8`). Existen dos casos posibles:

- Si el desplazamiento es menor o igual a 5, los 3 bits de la ficha caben completos dentro de un mismo byte, y basta con desplazar dicho byte hacia la derecha la cantidad de posiciones indicada por el desplazamiento y aplicar una máscara de 3 bits (`0x07`) para aislar el valor.
- Si el desplazamiento es mayor a 5 (es decir, 6 o 7), los 3 bits de la ficha quedan repartidos entre el byte actual y el siguiente. En este caso se extraen por separado los bits disponibles en el primer byte y los bits restantes en el segundo byte, y se combinan mediante desplazamientos y la operación OR bit a bit para reconstruir el valor completo de 3 bits.

La función de escritura sigue la misma lógica de localización, pero en lugar de extraer bits, primero limpia con una máscara invertida (`~`) la posición donde se va a escribir, y luego inserta el nuevo valor mediante la operación OR, preservando intacto el resto de los bits del byte que no pertenecen a la ficha modificada.

```
1 unsigned char leerFicha(const unsigned char* tablero, int fila, int
   columna, int columnas) {
2     int indice = calcularIndice(fila, columna, columnas);
3     int bit_inicial = indice * 3;
4     int byte_inicial = bit_inicial / 8;
5     int desplazamiento = bit_inicial % 8;
6
7     if (desplazamiento <= 5) {
8         // Caso 1: Los 3 bits est n en el mismo byte
9         unsigned char valor = (tablero[byte_inicial] >> desplazamiento)
          & 0x07;
10        return valor;
11    } else {
12        // Caso 2: La ficha est repartida entre dos bytes
13        int bits_primer_byte = 8 - desplazamiento;
14        int bits_segundo_byte = 3 - bits_primer_byte;
15    }
```

```

16     unsigned char parte1 = (tablero[byte_inicial] >> desplazamiento)
17     ;
18     unsigned char mascara_segundo = (1 << bits_segundo_byte) - 1;
19     unsigned char parte2 = tablero[byte_inicial + 1] &
20         mascara_segundo;
21     unsigned char valor = parte1 | (parte2 << bits_primer_byte);
22     return valor & 0x07;
23 }
24
25 void escribirFicha(unsigned char* tablero, int fila, int columna, int
26     columnas, unsigned char valor) {
27     valor &= 0x07;
28
29     int indice = calcularIndice(fila, columna, columnas);
30     int bit_inicial = indice * 3;
31     int byte_inicial = bit_inicial / 8;
32     int desplazamiento = bit_inicial % 8;
33
34     if (desplazamiento <= 5) {
35         // Caso 1: Toda la ficha cae en un solo byte
36         tablero[byte_inicial] &= ~(0x07 << desplazamiento);
37         tablero[byte_inicial] |= (valor << desplazamiento);
38     } else {
39         // Caso 2: La ficha est repartida entre dos bytes
40         int bits_primer_byte = 8 - desplazamiento;
41
42         unsigned char mascara_limpieza1 = (1 << bits_primer_byte) - 1;
43         tablero[byte_inicial] &= ~(mascara_limpieza1 << desplazamiento);
44         tablero[byte_inicial] |= ((valor & mascara_limpieza1) <<
45             desplazamiento);
46
47         int bits_segundo_byte = 3 - bits_primer_byte;
48         unsigned char mascara_limpieza2 = (1 << bits_segundo_byte) - 1;
49
50         tablero[byte_inicial + 1] &= ~mascara_limpieza2;
51         tablero[byte_inicial + 1] |= (valor >> bits_primer_byte) &
52             mascara_limpieza2;
53     }
54 }

```

3.2. Detección de combinaciones

El algoritmo de detección recorre el tablero fila por fila (para las combinaciones horizontales) y columna por columna (para las verticales), manteniendo un contador de fichas iguales consecutivas. Este contador representa la cantidad de fichas del mismo tipo encontradas seguidas hasta el momento: se incrementa mientras la ficha actual coincide con la siguiente, y se reinicia a 1 en cuanto cambia el tipo de ficha, pues en ese punto comienza una posible nueva combinación.

Al finalizar el recorrido de cada fila o columna es necesario realizar una comprobación adicional fuera del ciclo for. Esto se debe a que, si una combinación válida termina justo en la última posición del tablero (el borde), el ciclo concluye sin haber entrado nunca a la rama donde se evalúa si `contador >= 3`, por lo que esa combinación quedaría sin detectar si no se verificara explícitamente después del bucle.

```

1 int detectarCombinacionesCompacta(unsigned char* tablero, unsigned char*
2     marcas, int filas, int columnas)

```

```

2 {
3     int cantidadCombinaciones = 0;
4
5     // DETECCION HORIZONTAL
6     for(int f = 0; f < filas; f++)
7     {
8         int contador = 1;
9
10        for(int c = 0; c < columnas - 1; c++)
11        {
12            unsigned char actual = leerFicha(tablero, f, c, columnas);
13            unsigned char siguiente = leerFicha(tablero, f, c + 1,
14                columnas);
15
16            if(actual < 6 && actual == siguiente)
17            {
18                contador++;
19            }
20            else
21            {
22                if(contador >= 3)
23                {
24                    cantidadCombinaciones++;
25
26                    for(int k = 0; k < contador; k++)
27                    {
28                        marcarPosicion(marcas, f, c - k, columnas);
29                    }
30
31                    contador = 1;
32                }
33            }
34
35            // Combinacion al final de la fila
36            if(contador >= 3)
37            {
38                cantidadCombinaciones++;
39
40                for(int k = 0; k < contador; k++)
41                {
42                    marcarPosicion(marcas, f, columnas - 1 - k, columnas);
43                }
44            }
45        }
46
47        // DETECCION VERTICAL
48        for(int c = 0; c < columnas; c++)
49        {
50            int contador = 1;
51
52            for(int f = 0; f < filas - 1; f++)
53            {
54                unsigned char actual = leerFicha(tablero, f, c, columnas);
55                unsigned char siguiente = leerFicha(tablero, f + 1, c,
56                    columnas);
57
58                if(actual < 6 && actual == siguiente)

```

```

58     {
59         contador++;
60     }
61     else
62     {
63         if(contador >= 3)
64         {
65             cantidadCombinaciones++;
66
67             for(int k = 0; k < contador; k++)
68             {
69                 marcarPosicion(marcas, f - k, c, columnas);
70             }
71         }
72
73         contador = 1;
74     }
75 }
76
77 // Combinacion al final de la columna
78 if(contador >= 3)
79 {
80     cantidadCombinaciones++;
81
82     for(int k = 0; k < contador; k++)
83     {
84         marcarPosicion(marcas, filas - 1 - k, c, columnas);
85     }
86 }
87 }
88
89 return cantidadCombinaciones;
90 }

```

3.3. Aplicación de gravedad

El recorrido se realiza de abajo hacia arriba, columna por columna, utilizando una variable **destino** que indica la siguiente posición libre (desde el fondo del tablero) donde debe ubicarse una ficha válida. Cada vez que se encuentra una ficha distinta del estado vacío, esta se escribe en la posición **destino** y dicha variable se decrementa.

Recorrer el tablero en este sentido, de abajo hacia arriba, es necesario para no sobrescribir información que todavía no ha sido leída: como las fichas se desplazan hacia abajo, procesar primero las posiciones inferiores garantiza que, al momento de escribir en una posición, ya se haya leído previamente cualquier dato que dependiera de ella. Una vez recorrida toda la columna, las posiciones restantes por encima de **destino** se rellenan con el código de estado vacío.

```

1 void aplicarGravedad(unsigned char* tablero,int filas,int columnas)
2 {
3     for(int c = 0; c < columnas; c++)
4     {
5         int destino = filas - 1; //Si tengo Fila 0, fila1, fila2, fila3
6                                 //La fila del fondo es la fila 3, osea
7                                 //Necesario para empezar a recorrer las
8                                 //columnas desde abajo
9
9         for(int f = filas-1; f >= 0; f--)

```

```

10     {
11         unsigned char valor = leerFicha(tablero, f, c, columnas);
12
13         //Solo escribo en cada casilla si es una ficha valida, si es
14         un espacio no hago nada
15         if(valor != 6)
16         {
17             escribirFicha(tablero, destino, c, columnas, valor);
18
19             destino--;
20         }
21     }
22
23     //Reemplazo las posiciones de arriba con vacios
24     for(int f = 0; f <= destino; f++)
25     {
26         escribirFicha(tablero, f, c, columnas, 6);
27     }
28 }
29

```

3.4. Resolución de cascadas

Esta función implementa el ciclo que procesa automáticamente las combinaciones sucesivas generadas tras una eliminación, hasta que el tablero alcanza un estado estable sin combinaciones pendientes. En cada iteración del bucle se limpia primero la matriz de marcado (`limpiarMascara`), ya que las marcas de la iteración anterior corresponden a fichas que ya fueron eliminadas y no deben interferir con la detección de nuevas combinaciones producidas por el reacomodo del tablero.

La condición de parada del ciclo es que `detectarCombinaciones` retorne 0, es decir, que no se haya encontrado ninguna combinación nueva en esa vuelta. Mientras existan combinaciones, se acumulan las estadísticas del turno (cantidad de cascadas y de combinaciones), se eliminan las fichas marcadas, se aplica gravedad para desplazar las fichas restantes hacia abajo, y se rellenan los espacios vacíos con fichas nuevas generadas aleatoriamente, lo cual puede a su vez dar lugar a una nueva combinación en la siguiente iteración.

```

1  int resolverCascadas(unsigned char* tablero, bool** marcas, int filas,
2  int columnas, int& cascadasTurno, int& combinacionesTurno)
3  {
4      int totalEliminadasCascada = 0;
5      cascadasTurno = 0;
6      combinacionesTurno = 0;
7
8      while(true)
9      {
10         limpiarMascara(marcas, filas, columnas);
11
12         int encontradas = detectarCombinaciones(tablero, marcas, filas,
13         columnas);
14
15         if(encontradas == 0)
16         {
17             break;
18         }
19     }
20 }

```

```

18     combinacionesTurno += encontradas;
19     cascadasTurno++;
20
21     totalEliminadasCascada += eliminarMarcadas(tablero, marcas,
22         filas, columnas);
23
24     aplicarGravedad(tablero, filas, columnas);
25
26     rellenarTablero(tablero, filas, columnas);
27 }
28
29     return totalEliminadasCascada;
30 }

```

3.5. Modificación dinámica de la estructura (filas/columnas)

Cuando la eliminación de una fila o columna no reduce la utilización de memoria por debajo del 65 %, no se reserva un nuevo bloque de memoria; en su lugar, se reempaqueta la información directamente sobre el mismo bloque físico (*in-situ*). Esta operación exige cuidado, ya que implica leer y escribir simultáneamente sobre el mismo arreglo de bytes.

La seguridad de esta operación se sustenta en el orden de acceso: para cada posición se lee primero la ficha correspondiente a la posición siguiente (la que debe desplazarse) y, a continuación, se escribe dicho valor en la posición anterior (su nueva ubicación). Dado que el índice de escritura siempre es menor o igual al índice de lectura en el momento en que se procesa cada posición, la información que aún resta por leer no ha sido sobrescrita todavía cuando se necesita. De esta manera se garantiza que no se corrompan datos durante el reempaquetado, sin necesidad de reservar memoria adicional.

```

1 void eliminarColumna(unsigned char*& tablero, int filas, int& columnas,
2     int posicion, int& bytesReservados)
3 {
4     int columnasNuevas = columnas - 1;
5     int bytesNecesarios = calcularCantidadBytes(filas, columnasNuevas);
6     double utilizacion = (double)bytesNecesarios / bytesReservados;
7     bool reducirMemoria = utilizacion < 0.65;
8
9     if(reducirMemoria)
10     {
11         unsigned char* nuevoTablero = crearTablero(filas, columnasNuevas);
12
13         for(int f = 0; f < filas; f++)
14         {
15             for(int c = 0; c < columnas; c++)
16             {
17                 if(c == posicion)
18                     continue;
19
20                 int nuevaColumna;
21                 if(c < posicion)
22                     nuevaColumna = c;
23                 else
24                     nuevaColumna = c - 1;
25
26                 unsigned char ficha = leerFicha(tablero, f, c, columnas);
27
28                 escribirFicha(nuevoTablero, f, nuevaColumna, ficha);
29             }
30         }
31
32         delete[] tablero;
33         tablero = nuevoTablero;
34         bytesReservados = bytesNecesarios;
35     }
36 }

```

```

26         escribirFicha(nuevoTablero, f, nuevaColumna,
27                        columnasNuevas, ficha);
28     }
29 }
30
31 destruirTablero(tablero);
32 tablero = nuevoTablero;
33 bytesReservados = bytesNecesarios;
34 }
35 else
36 {
37     // Reempaquetado in-situ seguro
38     for(int f = 0; f < filas; f++)
39     {
40         for(int c = 0; c < columnas; c++)
41         {
42             if(c == posicion)
43                 continue;
44
45             int nuevaColumna;
46             if(c < posicion)
47                 nuevaColumna = c;
48             else
49                 nuevaColumna = c - 1;
50
51             unsigned char ficha = leerFicha(tablero, f, c, columnas)
52                                     ;
53             escribirFicha(tablero, f, nuevaColumna, columnasNuevas,
54                           ficha);
55         }
56     }
57
58     columnas = columnasNuevas;
59 }

```

4. Problemas de desarrollo que afrontó

1. **Comprensión del ordenamiento de bits en el empaquetado:** Aunque la primera versión funcional de `leerFicha` y `escribirFicha` operó correctamente desde el primer intento para el caso de fichas repartidas entre dos bytes, llegar a esa versión exigió un estudio y análisis considerable. La dificultad principal estuvo en comprender con precisión qué operaciones de desplazamiento y máscara eran necesarias para garantizar que el bit menos significativo de cada ficha quedara siempre alineado correctamente dentro de la secuencia de bits, de forma consistente tanto para el caso en que la ficha cabe completa en un byte como para el caso en que sus 3 bits se reparten entre dos bytes consecutivos.
2. **Detección incorrecta de combinaciones en L o en T:** En una versión intermedia se intentó marcar las fichas eliminadas sobrescribiendo directamente su valor en el tablero durante la fase de verificación por columnas. Este reemplazo inmediato causaba que, al llegar a la verificación vertical, ya no fuera posible identificar correctamente combinaciones con forma de T o de L, es decir, casos en los que una misma ficha forma parte simultáneamente de una combinación horizontal y de una vertical: al haberse modificado su valor durante el marcado horizontal, la comparación vertical posterior ya no encontraba la coincidencia original. Esto llevó a separar completamente la fase de detección de la fase de eliminación

mediante una estructura auxiliar de marcado, de forma que el tablero original permanece intacto durante todo el proceso de verificación.

5. Evolución de la solución y consideraciones para la implementación

5.1. Optimización de la estructura de marcado

La solución anterior utiliza una matriz `bool** marcas` del mismo tamaño que el tablero para separar la fase de detección de la fase de eliminación. Sin embargo, esta estructura resulta ineficiente en términos de memoria: reserva un byte completo por cada posición del tablero, mientras que el tablero mismo únicamente requiere 3 bits por posición. Como mejora prevista, se plantea reemplazar esta matriz por un arreglo compacto de bits, donde cada posición se representa con un único bit (1 si la ficha en esa posición fue marcada para eliminación, 0 en caso contrario), cuyo índice coincide con el índice lógico de la ficha correspondiente en el tablero. De esta forma, una combinación detectada se representaría, por ejemplo, como una secuencia de bits en 1 (111000...) en las posiciones marcadas, manteniendo la misma lógica de separación entre detección y eliminación pero con un costo de memoria proporcional al del propio tablero, en lugar de ser varias veces mayor.